

RAPPORT DE PROJET — AUTOSEMANTIC

1. Introduction et Objectifs

1.1 Vue d'ensemble

L'objectif du projet **AutoSemantic** est de concevoir, construire et exploiter un graphe de connaissances couvrant le domaine automobile. Le projet adresse la chaîne complète de valorisation de la donnée : de la collecte hétérogène (fichiers structurés et web scraping non structuré) jusqu'à l'interrogation sémantique et la recommandation.

1.2 Périmètre fonctionnel

Le système réalisé couvre les aspects suivants :

- **Extraction d'information** : Traitement de sources CSV officielles et scraping de données web enrichies par des LLM locaux (Ollama).
- **Lifting sémantique** : Mapping de données CSV et JSON vers RDF via RML.
- **Consolidation** : Fusion de graphes, validation SHACL et inférence OWL-RL.
- **Exploitation** : Un ensemble de requêtes SPARQL, un moteur de recommandation et deux approches RAG (Vectorielle vs Graph).

Note sur le Mode Test : Toutes ces valeurs sont en réalité 10% des valeurs totales, car nous avons mis en place un mode test pour ne garder qu'une partie des données, sinon cela prendrait trop de temps à tout exécuter. Il en est de même pour le scraping des données qui ne prend que quelques modèles et ne traite donc que quelques dizaines d'URL (au lieu de milliers).

2. Architecture des Données et Statistiques

Le graphe final est le résultat de l'agrégation de plusieurs sources hétérogènes.

2.1 Sources de Données

1. Données Structurées (CSV) :

- vehicles.csv (US Government, EPA) : 6 552 lignes, 79 colonnes. Contient les spécifications officielles (consommation, émissions).
- reviews_final.csv (Dataset Kaggle) : ~41 000 avis utilisateurs, incluant une note (1–5) et un texte libre.

2. Données Non Structurées (Web + JSON) :

- vehicles_scraped_clean.json : Données issues de **UltimateSpecs** pour 1 299 véhicules. Ce fichier est le résultat d'un processus de scraping suivi d'une structuration par LLM.

2.2 Volumétrie du Graphe (Mode Test)

Après le pipeline de traitement, les fichiers RDF générés sont les suivants :

- output.ttl (issu du CSV) : ~11,4 MB, **43 332 triplets**.
- kg_from_web_ollama.ttl (issu du JSON) : 23 KB, **467 triplets**.
- **knowledge_graph.ttl (Graphe fusionné final)** : 11,66 MB, contenant environ **43 799 triplets** et décrivant **7 243 entités** (dont 1 299 véhicules uniques).

3. Modélisation Sémantique et Validation

Le cœur du système AutoSemantic repose sur une ontologie formelle (OWL 2) couplée à des vocabulaires contrôlés (SKOS) et un jeu de règles de validation (SHACL). Cette structure garantit l'interopérabilité des données et permet d'automatiser la classification des véhicules tout en filtrant les incohérences.

3.1 Structure de l'Ontologie (autosemantic.ttl)

L'ontologie capture à la fois les spécifications techniques et l'expérience utilisateur.

- **Classes Principales (Squelette du graphe) :**
 - **Vehicle** (et sa sous-classe Car) : L'entité centrale.
 - **Manufacturer** : Les constructeurs automobiles.
 - **CustomerReview** : Les avis utilisateurs, reliés sémantiquement aux véhicules.
- Vocabulaires Contrôlés (SKOS) :

Pour éviter la prolifération de chaînes littérales et gérer les hiérarchies, des schémas de concepts ont été définis :

- FuelTypeScheme : Distingue les carburants fossiles (Essence, Diesel) des énergies propres (Électricité, Hydrogène).
- TransmissionScheme : Hiérarchise les boîtes de vitesses (ex: "Dual Clutch" est un broader concept de boîte automatique).
- VehicleClassScheme : Normalise la segmentation (SUV, Pickup, Compact, etc.).

3.2 Inférence et Classification Automatique

Le projet exploite la puissance des **Classes Définies OWL**. Le raisonneur classe dynamiquement les instances sans intervention humaine :

- **HighPerformanceCar** : Inférence basée sur la puissance. Définie comme une voiture (non utilitaire) possédant un moteur > 4.0L OU plus de 6 cylindres.
- **CityCar** : Intersection sémantique. Véhicule compact ou sous-compact avec une consommation urbaine stricte (< 9.4 L/100km).
- **OffRoadVehicle** : Inférence croisée entre la classe (Camion/SUV) et le type de transmission (4WD/AWD).
- **PositiveReview / CriticalReview** : Classification automatique des avis selon la note (≥ 4 ou ≤ 2).

3.3 Validation des Données (shapes.ttl)

Un schéma de validation **SHACL** agit comme un pare-feu sémantique pour garantir la robustesse du Knowledge Graph.

A. VehicleShape (Intégrité Technique)

Vérifie la cohérence physique et logique des données véhicules :

- **Logique Moteur** : Si fuelType est électrique, alors cylinders doit être 0 et la consommation d'essence nulle.
- **Incohérence Eco-Physique** : Rejet des aberrations (ex: moteur thermique > 4.0L déclarant une consommation < 5.0L/100km).
- **Cohérence de Marque** : Interdit à un LuxuryManufacturer de produire des véhicules utilitaires de base (ex: Cargo Van).
- **Contraintes de Valeurs** : Vérification des plages (Année 1886-2026, max 16 cylindres).

B. CustomerReviewShape (Qualité et Anti-Spam)

Assure la pertinence des avis collectés :

- **Règles Anti-Spam** : Une note parfaite (5/5) exige un texte > 20 caractères ; une critique (1/5) exige une explication > 15 caractères.
- **Avis Orphelins** : Détection des avis pointant vers des URIs de véhicules inexistantes.

4. Méthodologie de Construction (Pipeline ETL)

Le processus de construction du graphe est automatisé via un script global (run_pipeline.ps1).

4.1 Extraction et Nettoyage

Des scripts Python dédiés assurent la qualité des données en amont du lifting RDF :

- **Web Scraping & LLM (kg_ollama.py)** : Utilisation de BeautifulSoup4 pour le scraping et d'Ollama (modèle phi3) pour extraire les entités nommées (marque, modèle, année, cylindrée, type de motricité, etc.) depuis le texte brut des pages web.
- **Nettoyage JSON (clean_vehicles_json.py)** : Validation des types, filtrage des années aberrantes et normalisation des libellés (fuel, transmission) vers les concepts SKOS de l'ontologie.
- **Nettoyage CSV (clean_data.py)** : Harmonisation des types numériques, parsing des cylindrées, imputation des valeurs manquantes (médiane/mode) et suppression des colonnes redondantes.

4.2 Mapping RML (Lifting)

La transformation des données en triplets RDF est réalisée via **RML (RDF Mapping Language)**.

- **Mapping CSV** : 3_data_extraction/rml_mapping/mapping.ttl
- **Mapping JSON** : 3_data_extraction/web_extraction/mapping_json.ttl
- **Exécution** : Utilisation de l'image Docker rmlio/rmlmapper-java pour garantir la reproductibilité.

4.3 Fusion et Validation

- **Fusion (merge_graphs.py)** : Charge les graphes partiels (output.ttl, kg_from_web_ollama.ttl) et les alignements, lie les namespaces et produit le fichier final knowledge_graph.ttl.
- **Exécution de la Validation (validate_ontology.py)** : Ce script applique concrètement les règles SHACL et les inférences OWL décrites dans la Section 3, journalisant les violations pour correction itérative et produisant le graphe enrichi.

4.4 Alignements et Interopérabilité

Pour enrichir le graphe et l'ancrer dans le Web de Données (LOD), des alignements ont été réalisés (4_alignment/*.ttl) :

- **Vocabulaires cibles** : DBpedia, Wikidata, Schema.org.
- **Relations** : owl:sameAs pour les constructeurs majeurs ; skos:exactMatch pour les concepts techniques (carburants, transmissions).

5. Exploitation du Graphe

L'application n'est pas seulement un entrepôt de données, elle offre des services d'interrogation, de recommandation et d'IA générative.

5.1 Requêtes de Compétences (SPARQL)

Six requêtes complexes ont été implémentées (7_exploitation/sparql_queries) pour valider les capacités du graphe :

1. **Haute Performance** : Filtre les véhicules post-2000, triés par cylindrée et nombre de cylindres.
2. **Analyse de Sentiment** : Identification des véhicules ayant des avis positifs (Moyenne élevée avec ≥ 3 avis).
3. **Efficacité Énergétique** : Agrégation par constructeur pour comparer les performances écologiques.
4. **Capacités Off-Road** : Recherche ciblée sur les types de motricité (4x4) et classes (SUV).
5. **Véhicules de Luxe** : Croisement de critères de prix, puissance et options.
6. **Requête Fédérée (SERVICE)** : Enrichissement dynamique des données locales avec des informations provenant de DBpedia/Wikidata.

5.2 Système de Recommandation (Link Prediction)

Un module de recommandation (recommend.py) a été développé pour suggérer des véhicules similaires en utilisant une pondération sur les attributs (Marque, Année, Cylindrée, Conso, Note).

- **Utilisation :**

```
python 7_exploitation/link_prediction/recommend.py --top_k 10 --brand "Audi" --after_year 2010
```

5.3 Approches RAG (Retrieval-Augmented Generation)

Afin d'interroger le graphe en langage naturel et de pallier les limitations des LLMs (hallucinations, connaissances figées), deux architectures de RAG ont été implémentées et comparées.

A. RAG Vectoriel (Baseline)

Le script vector_rag_demo.py implémente une approche standard traitant le fichier RDF comme du texte brut.

- **Méthodologie** : Le fichier Turtle (TTL) est découpé en segments de texte (chunks de 500 caractères). Ces segments sont vectorisés via le modèle sentence-transformers/all-MiniLM-L6-v2 et indexés dans une base vectorielle **FAISS**.
- **Processus** : À chaque question utilisateur, le système recherche les segments les plus similaires sémantiquement (similitude cosinus) et les fournit comme contexte au LLM (**Ollama/Phi-3**).
- **Limites identifiées** : Cette méthode ignore la structure du graphe. Elle risque de découper des triplets RDF (séparant le sujet de son objet) ou de récupérer des passages syntaxiques du fichier (préfixes, définitions) sans pertinence pour la réponse, ce qui bruite le contexte fourni au LLM.

B. Graph RAG (Approche Sémantique Structurée)

Le script `graph_rag_demo.py` propose une approche "Graph-Native" qui respecte la structure ontologique des données.

- **Détection d'Intention** : Contrairement à la recherche vectorielle floue, ce système analyse la question pour identifier l'intention de l'utilisateur (ex : recherche de spécifications, info constructeur, avis clients) via des mots-clés.
- **Retrieval via SPARQL** : Au lieu de récupérer du texte, le système sélectionne dynamiquement un template SPARQL prédéfini correspondant à l'intention.
 - *Exemple* : Si l'utilisateur demande "Quelle est la consommation de la Ford Mustang ?", le script injecte l'entité "Ford Mustang" dans un template SPARQL ciblant les propriétés `:fuelConsumptionCity` et `:engineDisplacement`.
- **Génération de Réponse** : Les résultats exacts de la requête SPARQL sont formatés en un contexte factuel et précis, puis soumis au LLM.

C. Comparaison et Résultats

L'approche Graph RAG s'est révélée supérieure pour les requêtes factuelles :

1. **Précision** : Le Graph RAG ne fournit que les données demandées (ex: la valeur exacte de consommation), éliminant le risque d'hallucination induit par un contexte textuel trop large.
2. **Explicabilité** : La récupération de données est traçable via la requête SPARQL générée, contrairement à la "boîte noire" des embeddings vectoriels.
3. **Agrégation** : Seul le Graph RAG permet de répondre correctement à des questions nécessitant des agrégats (ex: "Quelle est la note moyenne ?") car il délègue le calcul au moteur SPARQL avant la génération de texte.

6. Interface et Exécution

L'application interagit avec l'utilisateur principalement via une interface en ligne de commande (CLI), orchestrée par un pipeline PowerShell.

Étapes exécutées séquentiellement :

1. Nettoyage des données sources.
2. Extraction web via Ollama (si JSON absent).
3. Génération RDF via Docker RMLMapper.
4. Fusion des graphes.
5. Validation SHACL et Inférence OWL.
6. Exécution des tests d'exploitation (Recommandation, SPARQL, RAG).

7. Conclusion

Le projet AutoSemantic a permis de valider la faisabilité d'une chaîne de traitement sémantique complète.

- **Réussites** : L'intégration de données non structurées via un LLM (Ollama) couplée à la rigueur du mapping RML a permis de créer un graphe cohérent (validé par SHACL) et riche (inférence OWL-RL).
- **Innovation** : La comparaison des approches RAG démontre que pour des données techniques structurées, l'approche Graph-RAG (SPARQL Templates) surpasse l'approche vectorielle naïve en précision et explicabilité.
- **Ouverture** : Grâce aux alignements SKOS et aux liens owl:sameAs, le graphe est prêt pour une intégration plus large dans le Linked Open Data.